a + b / c − d * 2 − e

d b w b 2

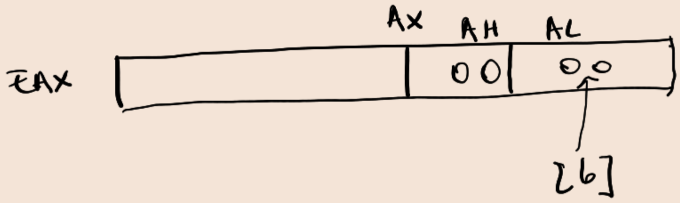
$\underbrace{\underbrace{b / c}_{w}}_{d}$

deîmp = dx : ax

împ = c

cât = AX

rest = DX



Extindere de la dw la dd : (word → dword)

unsigned:

1) movzx ebx, ax

⇕

2)
```
mov dx, 0
push dx
push ax
pop ebx
```

signed:

1) movsx ebx, ax

2) cwde ; eax = b/c

$\boxed{b/c}$ → b w

unsigned:

```
mov ax, 0
mov al, [b]
mov dx, 0
```
} dx : ax = b.

signed:

```
mov al, [b]
cbw      ; al → ax
cwd      ; ax → dx : ax
idiv word [c]   ; ax = b/c
```

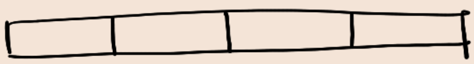
```
div word [c]   ; ax = b/c
add eax, [a]
mov ebx, eax   ; ebx = a + b/c
mov al, 2
mul byte [d]   ; ax = d*2
```

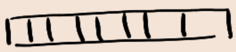
imul byte [d]

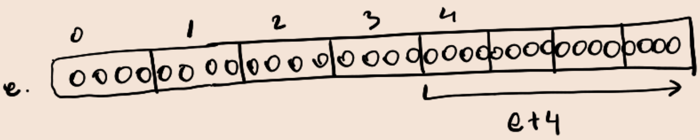
```
mov dx, 0
push dx
push ax
pop eax
```
ax → eax ; eax = d*2 <=> cwde

```
sub ebx, eax   ; a+b/c − d*2
mov eax, ebx
mov edx, 0
```
<=> cdq

```
sub eax, [e]
sbb edx, [e+4]
```

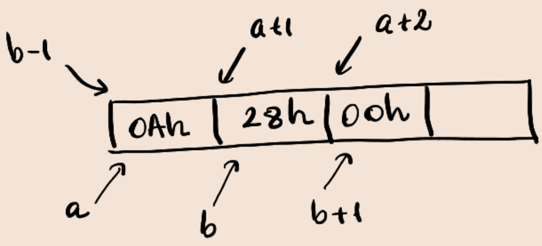






a db 10
b dw 40

$10 = 0Ah$
$40 = 28h = 0028h$



UNSIGNED

Împrumut

$$\begin{array}{r} 1\ 0110\ 0010b - \\ 1100\ 1000b \\ \hline 1001\ 1010b \end{array}$$

$\rightarrow 0110\ 0010b = 1\cdot 2^1 + 1\cdot 2^5 + 1\cdot 2^6 = 64 + 32 + 2 = 98$

$\rightarrow 1100\ 1000b = 1\cdot 2^7 + 1\cdot 2^6 + 1\cdot 2^3 = 128 + 64 + 8 = 200$

$\rightarrow 1001\ 1010b = 1\cdot 2^7 + 1\cdot 2^4 + 1\cdot 2^3 + 1\cdot 2 = 154.$

$\Updownarrow$

„AR TREBUI"

$$\begin{array}{r} 98 - \\ 200 \\ \hline 154 \end{array}$$ $\underbrace{\text{Fals, } \neq -102}_{CF = 1.}$

SIGNED

$$\begin{array}{r} 98 - \\ -56 \\ \hline -102 \end{array}$$ Fals!

$\underbrace{\neq 154}_{OF = 1}$

bit de semn 0

$0110\ 0010b = 98$

$1100\ 1000b = -56$

bit de semn 1

↳ abs: 2'scomp: $00111000b$

$= 1\cdot 2^3 + 1\cdot 2^4 + 1\cdot 2^5 = 56$

$1001\ 1010b = -102$

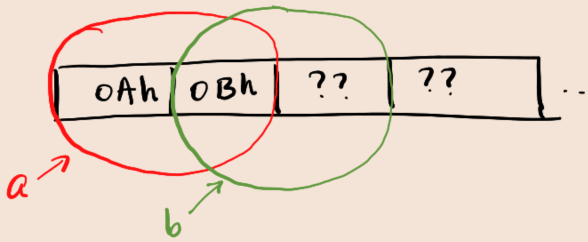
↳ abs: 2'scomp: $0110\ 0110b$

$= 1\cdot 2^1 + 1\cdot 2^2 + 1\cdot 2^5 + 1\cdot 2^6$

$= 64 + 32 + 4 + 2$

$= 102$

a db 10
b db 11



} de ce e corectă sintactic, dar incorectă logic.

```
mov AX, [a] ; AX = 0B0Ah
add AX, [b] ; AX = 0B0Ah + ??0Bh

segment data use32 class=data
    a dw 0111011101000111b
    b dw 1001101110111110b
    c dw 0

segment code use32 class=code
start:
    mov bx,0
    mov ax, [b]
    and ax, 0001110000000000b
    mov cl,10
    ror ax,cl
    or bx,ax

    or bx, 0000000001111000b

    mov ax, [a]
    and ax, 0000000000011110b
    mov cl, 6
    rol ax,cl
    or bx,ax

    and bx,1110011111111111b

    mov ax,[b]
    not ax
    and ax,0000111000000...b
    mov cl,4
    rol ax,cl
    or bx,ax

    mov [c], bx
```

ax: 1001101110111110b = 9BBEh
9 B B E

AX: | 9B | BE |

```
ax: 1001101110111110
    0001110000000000   AND
    ----------------
ax: 0001100000000000
```

AX: | 18 | 00 |

CX: | | OA | (CL)

ax: 0000 0000 0000 0110

AX: | 00 | 06 |

BX: | 00 | 06 |

```
BX: 0000 0000 0000 0110
    0000 0000 0111 1000   OR
    -------------------
BX: 0000 0000 0111 1110
```

DIV BX ; DX:AX / BX = AX (quotient)
DX:AX % BX = DX (remainder)

DIV BL ; AX / BL = AL (quotient)
AX % BL = AH (remainder)

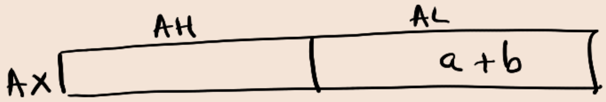
MOV AX, 5
MOV CL, 2

DIV CL

1. ((a+b)*c)/d , unsigned . a, b, c, d bytes

segment data use 32 ...
a db ...
b db ...
c db ...
d db ...

segment code ....
start :



mov al, [a] ; al ← a
add al, [b] ; al ← a+b
mul byte [c] ; ax ← al * [c] = (a+b)*c
div byte [d] ; $\overset{c}{\text{al}}$, $\overset{r}{\text{ah}}$ ← ax / d
mov [x], al ;
push dword [0]
call [exit]